Week 10 Lab: SPI Protocol & DMA — W25Q Flash

EE0827 Embedded System Applications

Overview

Talk to a Winbond W25Q SPI flash from an STM32 Nucleo. You will read the JEDEC ID, erase a sector, write data, read it back, and then convert your polled driver to a DMA-driven one — all while watching the four SPI signals on a logic analyzer.

Time allocation: 90 minutes

- Hardware & CubeMX setup: 15 min
- JEDEC ID + polled read: 25 min
- Erase + Program + Read pattern: 25 min
- Convert to DMA + logic analyzer capture: 25 min

Learning Objectives

1. Wire a 4-wire SPI device (SCK/MOSI/MISO/CS) and verify the bus on a scope or LA
2. Configure STM32 SPI1 in CubeMX and pick the right mode (CPOL/CPHA) from a datasheet
3. Use the W25Q command set: JEDEC ID, Write Enable, Page Program, Sector Erase, Read
4. Convert a working polled driver to DMA and verify CPU is freed during the transfer
5. Capture and decode SPI traffic with a logic analyzer

Safety & Setup

Safety Checklist

- ☐ USB cable connected to Nucleo board (USB power only)
- ☐ W25Q VCC on **3.3 V** rail — **the chip is 3.3 V only and 5 V will destroy it**
- ☐ **MOSI/MISO not swapped** (very common mistake; `DI` on the chip is master's MOSI)
- ☐ WP and HOLD pins tied to **3.3 V** (disable hardware write protect and clock pause)
- ☐ Logic analyzer ground connected to circuit ground

Hardware: Nucleo-F401RE, W25Q SPI flash breakout (any size: W25Q16/32/64/128), breadboard, jumper wires, Saleae-clone logic analyzer (or 4-channel oscilloscope).

Software: STM32CubeIDE, serial terminal (115200 baud), PulseView.

Pre-Lab Questions

1. SPI is full-duplex. To read 3 bytes from a slave, how many bytes must the master clock out, and what value should they be? _____

2. The W25Q datasheet says it supports SPI mode 0 and mode 3. What CPOL/CPHA values must you set for each, and what does each combination mean physically? _____

3. Why does every W25Q write or erase command have to be preceded by a 0x06 (Write Enable) command in its own CS-LOW window? _____

4. You configure DMA TX with peripheral-increment = ON by mistake. What happens to the SPI peripheral and to the rest of the chip? _____

Part 1

Hardware & CubeMX Setup

15 minutes

Step 1.1: Wire the W25Q Breakout

W25Q Pin	Nucleo Pin	Notes
VCC	3.3 V	CN6 Pin 4 — **3.3 V ONLY**
GND	GND	CN6 Pin 6 — common ground critical
CS	PB6	GPIO output, software-controlled
CLK	PA5	SPI1_SCK
DI	PA7	SPI1_MOSI (master out → chip's data in)
DO	PA6	SPI1_MISO (chip's data out → master in)
WP	3.3 V	Disable hardware write-protect
HOLD	3.3 V	Disable clock-pause

Logic analyzer: CH0 → CLK (PA5), CH1 → DI (PA7), CH2 → DO (PA6), CH3 → CS (PB6), GND → board GND.

Why CS on a regular GPIO and not the SPI peripheral?

The STM32 SPI peripheral has a built-in NSS pin, but it has well-known quirks (it pulses between bytes by default, which breaks multi-byte commands on most flashes). The robust pattern is to drive CS as a plain GPIO so you control exactly when it falls and rises. Every embedded codebase you'll see in the wild does it this way.

Powered check (3.3 V applied, bus idle, before flashing any code):

1. CS pin to GND reads ≈ 3.3 V (idle HIGH)
2. CLK pin to GND reads ≈ 0 V (CPOL=0 idle, default after reset)

Step 1.2: CubeMX Project

1. **New project** → Board: NUCLEO-F401RE → Name: `W10_SPI_Flash`
2. **Connectivity → SPI1**: Mode = Full-Duplex Master
   - Frame Format: Motorola, Data Size: 8 Bits, First Bit: MSB First
   - **Clock Polarity: Low** (CPOL=0), **Clock Phase: 1 Edge** (CPHA=0) — this is mode 0
   - Prescaler: 64 → SCK ≈ 1.3 MHz (slow for easy debugging)
   - NSS Signal Type: **Disable** (we drive CS in software)
3. **GPIO → PB6**: Output, Push-Pull, High-speed, **default level HIGH**, label `CS`

4. **Connectivity → USART2**: Asynchronous, 115200 baud, 8N1
5. **Generate code** and open in CubeIDE.

Part 2

JEDEC ID — The SPI WHO_AM_I

25 minutes

Step 2.1: Defines, printf Redirect, CS Helpers

Inside `main.c`:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define CS_LOW()  HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET)
#define CS_HIGH() HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET)

#define CMD_JEDEC_ID    0x9F
#define CMD_READ        0x03
#define CMD_PAGE_PROG   0x02
#define CMD_SECTOR_ERA  0x20
#define CMD_WRITE_EN    0x06
#define CMD_READ_STATUS 0x05

#define MFG_WINBOND     0xEF
/* USER CODE END PD */

/* USER CODE BEGIN 0 */
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
/* USER CODE END 0 */
```

Keep your code inside USER CODE markers!

CubeMX **deletes** everything outside `USER CODE BEGIN/END` blocks when you regenerate. Always place your code inside these markers.

Step 2.2: Read JEDEC ID

In `main()` after peripheral init:

```c
/* USER CODE BEGIN 2 */
printf("\r\n=== W10: SPI Flash Lab ===\r\n");

uint8_t tx[4] = { CMD_JEDEC_ID, 0xFF, 0xFF, 0xFF };
uint8_t rx[4] = { 0 };

CS_LOW();
HAL_SPI_TransmitReceive(&hspi1, tx, rx, 4, HAL_MAX_DELAY);
CS_HIGH();

printf("JEDEC ID: %02X %02X %02X (mfg %s)\r\n",
       rx[1], rx[2], rx[3],
       (rx[1] == MFG_WINBOND) ? "Winbond OK" : "UNKNOWN");
/* USER CODE END 2 */
```

Build, flash, open terminal at 115200 baud, press reset.

Expected output (the device-ID bytes vary by capacity):

```
=== W10: SPI Flash Lab ===
JEDEC ID: EF 40 15 (mfg Winbond OK)
```

Why discard rx[0

?] Recall that SPI is full-duplex: every SCK edge shifts both directions. When the master sends `0x9F` on the first byte, the slave is sending whatever was previously in its shift register — usually `0xFF` or stale data. The slave only *starts* producing the JEDEC ID on the next byte. So `rx[0]` is always meaningless on command-style reads. We discard it; `rx[1..3]` carries the real response.

Checkpoint 1

You must see `rx[1] = 0xEF` (Winbond manufacturer ID) before continuing. Without this, your bus is broken — see **Common Issues** on the last page. `rx[2]` and `rx[3]` encode capacity (e.g. `0x40 0x15` = W25Q16, `0x40 0x17` = W25Q64).

Part 3

Erase, Program, Verify

25 minutes

Step 3.1: Helper Functions

Add **above** `main()`:

```
/* USER CODE BEGIN 4 */
static void flash_write_enable(void) {
    uint8_t cmd = CMD_WRITE_EN;
    CS_LOW();
    HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);
    CS_HIGH();
}

static void flash_wait_ready(void) {
    uint8_t tx[2] = { CMD_READ_STATUS, 0xFF };
    uint8_t rx[2];
    do {
        CS_LOW();
        HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, HAL_MAX_DELAY);
        CS_HIGH();
    } while (rx[1] & 0x01);   // BUSY bit is bit 0 of status reg 1
}

static void flash_sector_erase(uint32_t addr) {
    uint8_t tx[4] = { CMD_SECTOR_ERA,
                      (addr >> 16) & 0xFF,
                      (addr >>  8) & 0xFF,
                       addr        & 0xFF };
    flash_write_enable();
    CS_LOW();
    HAL_SPI_Transmit(&hspi1, tx, 4, HAL_MAX_DELAY);
    CS_HIGH();
    flash_wait_ready();        // ~50 ms
}

static void flash_page_program(uint32_t addr, const uint8_t *data, uint16_t n) {
    uint8_t hdr[4] = { CMD_PAGE_PROG,
                       (addr >> 16) & 0xFF,
                       (addr >>  8) & 0xFF,
```

```
                          addr        & 0xFF };
    flash_write_enable();
    CS_LOW();
    HAL_SPI_Transmit(&hspi1, hdr, 4, HAL_MAX_DELAY);
    HAL_SPI_Transmit(&hspi1, (uint8_t*)data, n, HAL_MAX_DELAY);
    CS_HIGH();
    flash_wait_ready();        // ~1 ms
}

static void flash_read(uint32_t addr, uint8_t *buf, uint16_t n) {
    uint8_t hdr[4] = { CMD_READ,
                       (addr >> 16) & 0xFF,
                       (addr >>  8) & 0xFF,
                        addr        & 0xFF };
    CS_LOW();
    HAL_SPI_Transmit(&hspi1, hdr, 4, HAL_MAX_DELAY);
    HAL_SPI_Receive(&hspi1, buf, n, HAL_MAX_DELAY);
    CS_HIGH();
}
/* USER CODE END 4 */
```

Step 3.2: Erase, Program, Read Cycle

In `main()` **after** the JEDEC ID print:

```
const uint32_t TEST_ADDR = 0x000000;
const char *msg = "EE0827 W10 SPI Flash";

printf("Erasing sector 0...\r\n");
flash_sector_erase(TEST_ADDR);

uint8_t empty[32];
flash_read(TEST_ADDR, empty, sizeof empty);
printf("After erase, byte 0 = 0x%02X (expect 0xFF)\r\n", empty[0]);

printf("Writing test message...\r\n");
flash_page_program(TEST_ADDR, (const uint8_t*)msg, strlen(msg));

uint8_t back[32] = {0};
flash_read(TEST_ADDR, back, strlen(msg));
back[strlen(msg)] = 0;     // null-terminate
```

```
printf("Read back: \"%s\"\r\n", back);
printf("%s\r\n",
       (memcmp(msg, back, strlen(msg)) == 0) ? "VERIFY OK" : "VERIFY FAIL");
```

Expected output:

```
Erasing sector 0...
After erase, byte 0 = 0xFF (expect 0xFF)
Writing test message...
Read back: "EE0827 W10 SPI Flash"
VERIFY OK
```

Why does erase return 0xFF?

A flash erase sets every bit in the sector to 1 — so every byte reads as `0xFF`. Programming can only flip 1 bits to 0; you cannot turn a 0 back into a 1 without erasing the whole sector. This is why flash file systems are so much more complex than RAM data structures: every "modify" is really an "erase + reprogram everything you wanted to keep."

Checkpoint 2

You must see `VERIFY OK` before continuing. If the readback is the old data (or 0xFF), you forgot Write Enable or didn't wait for the busy bit to clear.

Part 4

DMA Conversion & Logic Analyzer *25 minutes*

Step 4.1: Add DMA in CubeMX

1. Reopen the `.ioc` file
2. **SPI1 → DMA Settings tab → Add** (twice):
   - First: **SPI1_TX**, Direction = Memory to Peripheral, Mode = **Normal**, Data Width = Byte, Memory Increment = **Enable**, Peripheral Increment = **Disable**
   - Second: **SPI1_RX**, Direction = Peripheral to Memory, same settings
3. **NVIC tab → Enable SPI1 global interrupt** (and DMA stream interrupts if not auto-enabled)
4. **Generate code** — answer "Yes" to "keep user code"

Most common DMA gotcha

Memory Increment must be **Enable** (the buffer needs to walk forward), and Peripheral Increment must be **Disable** (the SPI data register is one fixed location). Getting these backwards either freezes the chip or fills your buffer with the same byte 1024 times.

Step 4.2: DMA Read with Callback

Replace the JEDEC ID block in `main()` with this DMA version:

```
/* USER CODE BEGIN 4 (add) */
volatile uint8_t spi_busy = 0;
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi) {
    if (hspi->Instance == SPI1) {
        spi_busy = 0;
    }
}
/* USER CODE END 4 */

/* USER CODE BEGIN 2 (add after polled JEDEC ID) */
printf("\r\n--- DMA JEDEC ID test ---\r\n");
uint8_t dtx[4] = { CMD_JEDEC_ID, 0xFF, 0xFF, 0xFF };
uint8_t drx[4] = { 0 };

uint32_t blink_count = 0;
spi_busy = 1;
```

```
CS_LOW();
HAL_SPI_TransmitReceive_DMA(&hspi1, dtx, drx, 4);

// CPU is FREE during the transfer --- prove it by toggling the LED
while (spi_busy) {
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    blink_count++;
}
CS_HIGH();
printf("DMA JEDEC ID: %02X %02X %02X | LED toggled %lu times during transfer\r\n",
       drx[1], drx[2], drx[3], blink_count);
/* USER CODE END 2 */
```

Expected output (blink count varies; the point is it's $\gg 0$):

```
--- DMA JEDEC ID test ---
DMA JEDEC ID: EF 40 15 | LED toggled ~2400 times during transfer
```

What this proves

With polled SPI, the CPU is blocked inside `HAL_SPI_TransmitReceive` until the last byte clocks out — the LED would toggle exactly zero times. With DMA, the transfer happens entirely in hardware while the CPU runs the toggle loop. This is the entire reason DMA exists. Every audio decoder, video frame buffer, and ADC streaming app you'll ever write depends on this trick.

Step 4.3: Capture SPI Traffic on the Logic Analyzer

1. PulseView → Sample rate **8 MHz**, 100 ms capture
2. Add the **SPI** decoder: CLK=D0, MOSI=D1, MISO=D2, CS=D3
3. In the decoder options: **CPOL=0, CPHA=0, MSB-first, 8 bits, CS active LOW**
4. Start capture, press reset on Nucleo, stop capture

Identify on the trace:

- CS falls → MOSI sends `0x9F` → MISO sends 3 ID bytes → CS rises
- A separate later cycle shows the erase command (`0x20` + 3 address bytes)
- A long quiet gap ($\approx$ 50 ms) follows erase — that's `flash_wait_ready` polling
- The page-program command (`0x02` + 3 address + 20 data bytes) follows
- A second 1 ms quiet gap, then the read-back

Measurement	Expected	Measured
SCK frequency	$\approx$ 1.3 MHz	_____ MHz
Single byte time at 1.3 MHz	$\approx$ 6 µs	_____ µs
JEDEC ID full transfer (4 bytes)	$\approx$ 24 µs	_____ µs
CS-LOW to first SCK edge	$<$ 1 µs	_____ µs
Sector-erase wait time	$\approx$ 30–80 ms	_____ ms

Save annotated screenshots of: (a) the JEDEC ID transaction, (b) the erase-then-wait sequence.

Step 4.4: Failure Injection

For each test: **predict**, **test**, **restore**.

Test 1 — Wrong CPHA. In CubeMX, change Clock Phase to "2 Edge" (CPHA=1). Regenerate, flash. **Predict** what JEDEC ID you'll see: _____**Observed:** **Restore.**

Test 2 — Skip Write Enable. Comment out the `flash_write_enable()` call inside `flash_page_program()`. **Predict:** does the readback show the new message or the old erased data? _____**Observed:** _____**Restore.**

Test 3 — CS held LOW between command and data. Move the `CS_HIGH()` of `flash_page_program` to *after* `flash_wait_ready()` so CS stays low during the busy poll. **Predict:** what does the status-register read return? _____**Observed:** _____**Restore.**

Measurement Summary

ID	Measurement	Expected	Measured	Pass
M1	CS idle voltage (powered, before any code)	$\approx$ 3.3 V		☐
M2	JEDEC ID byte 1 (manufacturer)	0xEF		☐
M3	After-erase byte	0xFF		☐
M4	Readback verify	matches written		☐
M5	DMA JEDEC ID returns same value as polled	yes		☐
M6	LED toggle count during DMA transfer	$\gg$ 0		☐
M7	SCK frequency on LA	$\approx$ 1.3 MHz		☐

ID	Measurement	Expected	Measured	Pass
M8	Sector-erase wait time	$\approx$ 30–80 ms		☐

Analysis Questions

1. SPI is full-duplex but most of our reads only care about one direction. Why is full-duplex still a useful property — give one concrete example from this lab. _

2. We use a 64-divider prescaler ($\approx$ 1.3 MHz) for this lab. The W25Q supports 104 MHz. Name two reasons not to crank the SCK to 25 MHz on a breadboard.

3. The DMA version freed the CPU during a 4-byte transfer. For a 4-byte transfer at 1.3 MHz, the CPU saved $\approx$ 25 µs. Is that meaningful? When does DMA's value actually become large?

4. What would happen if you tried to call `HAL_SPI_TransmitReceive_DMA()` on a buffer declared on the stack inside a function that returns before the transfer completes?

Deliverables

Completion Checklist

- ☐ JEDEC ID returns `0xEF` as the first byte
- ☐ Erase → Program → Read returns the original message (`VERIFY OK`)
- ☐ DMA version returns the same JEDEC ID as the polled version
- ☐ LED toggle count $\gg 0$ during the DMA transfer
- ☐ Annotated logic-analyzer screenshots (JEDEC ID + erase-wait)
- ☐ Measurement table completed
- ☐ Failure injection tests documented
- ☐ Analysis questions answered

This lab is **ungraded practice**. Keep your code — you will reuse the W25Q driver in W12 (fault logging).

Common Issues

Issue	Fix
JEDEC ID = 0xFF 0xFF 0xFF	MISO not connected, or CS never asserted
JEDEC ID = 0x00 0x00 0x00	Chip unpowered or VCC short
JEDEC ID looks "shifted"	Wrong CPHA — try the other value
Erase returns 0xFF but program does nothing	Forgot Write Enable
Readback is original (pre-erase) data	Didn't wait for busy bit
Hard fault during DMA	Buffer on stack, gone before TC
DMA callback never fires	NVIC SPI1 interrupt not enabled
LED toggle count is 0 with DMA	Used polled API by mistake
All-zero readback after program	Forgot to wake from default protected state

Key Takeaways

1. **JEDEC ID first** — the W10 sanity check. If `0xEF` doesn't come back, the bus is broken; debug hardware before code.

2. **Discard rx[0] on command reads** — the slave's first byte out is always stale; useful response starts at rx[1].
3. **CS is software** — a GPIO you control byte-by-byte beats the SPI peripheral's NSS pin every time.
4. **Erase → Program is the rule** — flash bits go 1$\rightarrow$0$freelybutonlyerasecanflip0\rightarrow$\$1, and only by the sector.
5. **DMA = same code, different function name** — converting from polled to DMA is a one-line change once the IOC is set up. Memorize that pattern.